



2 📖
18 📄
2160 sp

نظري

كلية الهندسة المعلوماتية
السنة الخامسة - هندسة البرمجيات



Creational & Structural Pattern

د. باسم قصيبة

RB Informatics;

14/10/2024

هندسة برمجيات ٣

تحدثنا في المحاضرة السابقة عن ال Visitor وهو Behavioral Design Pattern وفي هذه المحاضرة سنأخذ أنواع أخرى من ال Design Patterns تندرج تحت Creational Design Patterns و Structural Design Patterns.

إحدى أعراض النظام السيء هي ال Dependency العالية وتظهر خاصة في تعليمة new لأنها تعتمد على كلاس حقيقي للإنشاء منه (new هي أسوأ Dependency).

Creational patterns

تهتم بعملية إنشاء الأغراض أي من يستطيع إنشاء الأغراض ومتى وأين ، ويندرج ضمنها:

- Singleton
- Abstract Factory
- Builder
- Factory Method
- Prototype

Singleton

يضمن وجود غرض (instance) واحد فقط من كلاس معين، مع توفير نقطة وصول عامة لهذا الغرض.

شروط التنفيذ:

- نجعل ال Constructor (الباني) private.
- نبني ال instance بنفس الكلاس.
- ننشئ method للوصول لهذا ال instance.

في الجافا قبل 1.5 لم تكن ال enum معرفة فتم استخدام بنية singleton لتمثيل هذه البنية.



```
public class SingletonClass {
    private SingletonClass() {}
    private static SingletonClass uniqueInstance = new SingletonClass();
    public static SingletonClass getTheInstance() {

        return uniqueInstance;
    }
}
```

مثال : تمثيل أيام الأسبوع

```
public class DaysOfWeek {
    public static final int LUNDI = 0;
    public static final int MARDI = 1;
    public static final int MERCREDI = 2;
    public static final int JEUDI = 3;
    public static final int VENDREDI = 4;
    public static final int SAMEDI = 5;
    public static final int DIMANCHE = 6;
}
```

Void methodProcessingDays(int day){...}

Obj.methodProcessingDays(DaysOfWeek.LUNDI);
Obj.methodProcessingDays(7 + 5);

Disadvantages: no type for the days of week but integer values.

The correct solution must : typed and constant values & number of these values is fixed.

في البنية السابقة تم تمثيل الأيام بقيم صحيحة و تم تمرير قيمة صحيحة كبارامتر لـ mehtodProcessingDays. تمرير القيمة 12 كبارامتر لـ mehtodProcessingDays ضمن ال compile Time لا تظهر أي مشاكل لكن تظهر ضمن ال RunTime ، في هذا الحل لم نستعمل بنية ال Singleton.

تَعَلَّم الإِتِّمَامَ ..

أَنْ تَتَمَّ مَا بَدَأْتَ، وَتُنْجِزَ مَا خَطَّوْتَ

```

public class DaysOfWeek {
    private DaysOfWeek() {}
    public static final DaysOfWeek LUNDI = new DaysOfWeek();
    public static final DaysOfWeek MARDI = new DaysOfWeek();
    public static final DaysOfWeek MERCREDI = new DaysOfWeek();
    public static final DaysOfWeek JEUDI = new DaysOfWeek();
    public static final DaysOfWeek VENDREDI = new DaysOfWeek();
    public static final DaysOfWeek SAMEDI = new DaysOfWeek();
    public static final DaysOfWeek DIMANCHE = new DaysOfWeek();
}

Void methodProcessingDays(DaysOfWeek day){...}

```

Advantages

- Days are only objects of the class DaysOfWeek.
- We can compare the days using "==" because each day has unique referece during all the life of the program.
- Days are only the instances created in the class DaysOfWeek.

ضمن الـ methodProcessingDays قمنا بتمرير object من نمط DaysOfWeeks (حصرا ضمن الأغراض الـ 7 المعرفة ببنية الكلاس).

الأيام عبارة عن object من كلاس DaysOfWeek، من الممكن المقارنة بين الأيام لان كل يوم له reference فريد طيلة حياة البرنامج.

تصوير لتمثيل الـ Enum ضمن الجافا:

فعليا ضمن البنية التالية قمنا ببناء غرض من نمط DaysOfWeeks وقمنا بإعطائه رقم، وضمن التابع compareTo قمنا بقصر الـ object على نمط DaysOfWeeks وقابلناه بالـ index الخاص فيه.

```

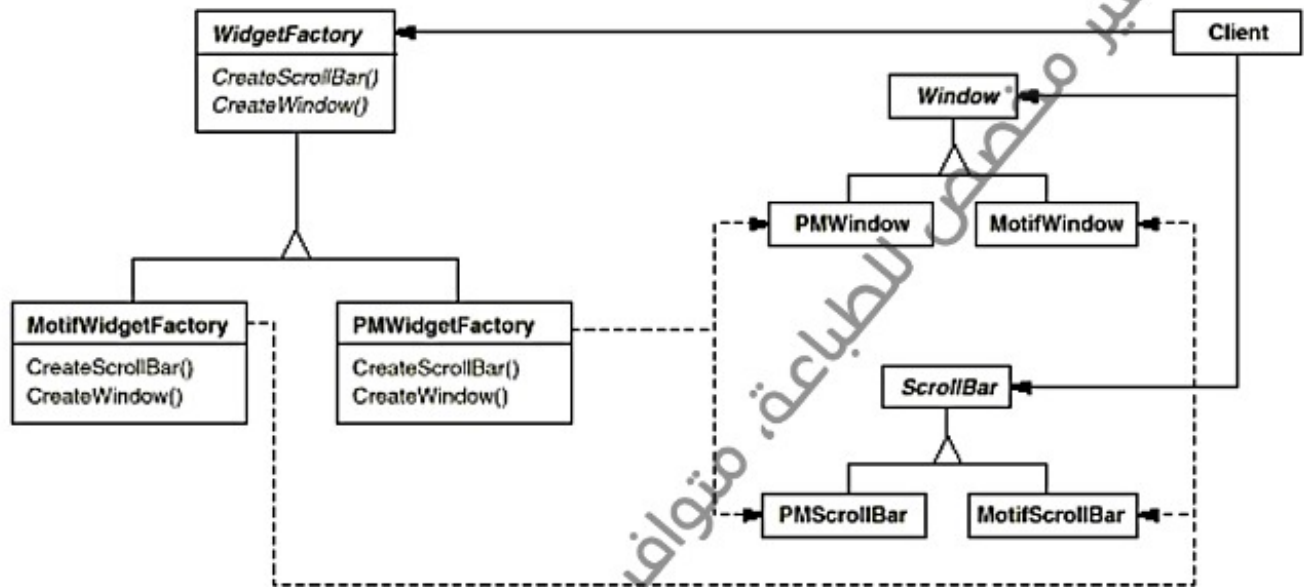
public class DaysOfWeek implements Comparable {
    private String name;
    private static int counter = 0;
    public final int index; // final so is not modifiable
    private DaysOfWeek(String name) {
        this.name = name;
        index = counter++;
    }
    public int compareTo(Object o) {
        DaysOfWeek j = (DaysOfWeek) o;
        return this.index - j.index;
    }
    public static final DaysOfWeek LUNDI = new DaysOfWeek("Lundi");
    ....
    public String toString() {return this.name;}
}

```

Abstract Factory

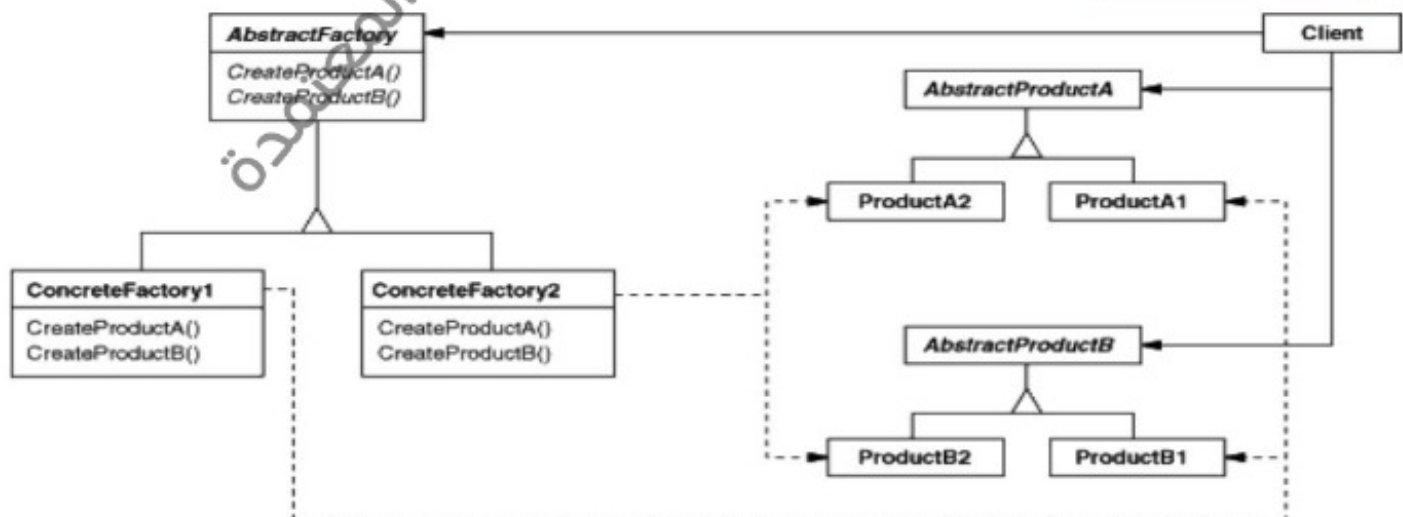
يؤمن interface لإنشاء عائلات ل objects مترابطة أو مجموعة من الكائنات بدون التعامل مع الصفوف الحقيقية، تم بناء ال abstract factory للانتقال من عائلة ما إلى عائلة أخرى بسهولة، عند عملية البناء نقوم بتمرير وسيط يحدد ما نرغب ببنائه.

■ مثال 1:



ضمن ال widgetFactory_interface نقوم بتعريف create methods لكل المكونات التي لدينا، ويرث من هذه ال interface عائلتان (motifwidgetFactory , PMwidgetFactory) تقومان داخلهما بتطبيق ال create methods المعرفة ضمن ال interface، createScrollBar() يقوم بعمل new ل ScrollBar و createWindow() يقوم بعمل new ل Window ، وذلك وفق التطبيق الملائم لكل عائلة. ال Client يتعامل مع ال AbstractFactory وحسب البارامتر الممرر يتم الانتقال بين العائلات.

بنية ال Abstract Factory:



```

interface MazeFactory {

    Maze MakeMaze();

    Wall MakeWall();

    Room MakeRoom(int n);

    Door MakeDoor(Room r1, Room r2);

}

class OrdinaryMazeFactory implements MazeFactory {

    public MazeFactory();

    Maze MakeMaze() { return new Maze(); }

    Wall MakeWall() { return new Wall(); }

    Room MakeRoom(int n) { return new Room(n); }

    Door MakeDoor(Room r1, Room r2)

    { return new Door(r1, r2); } }

class BombedMazeFactory implements MazeFactory {

    Wall MakeWall() { return new BombedWall(); }

    Room MakeRoom(int n) { return new BombedRoom(n);

    ....}

Class MazeGame {

Maze CreateMaze (MazeFactory factory) {

    Maze aMaze = factory.MakeMaze();

    Room r1 = factory.MakeRoom(1);

    Room r2 = factory.MakeRoom(2);

    Door aDoor = factory.MakeDoor(r1, r2);

    aMaze.AddRoom(r1);

    aMaze.AddRoom(r2);

    .....

    return aMaze; } }

```



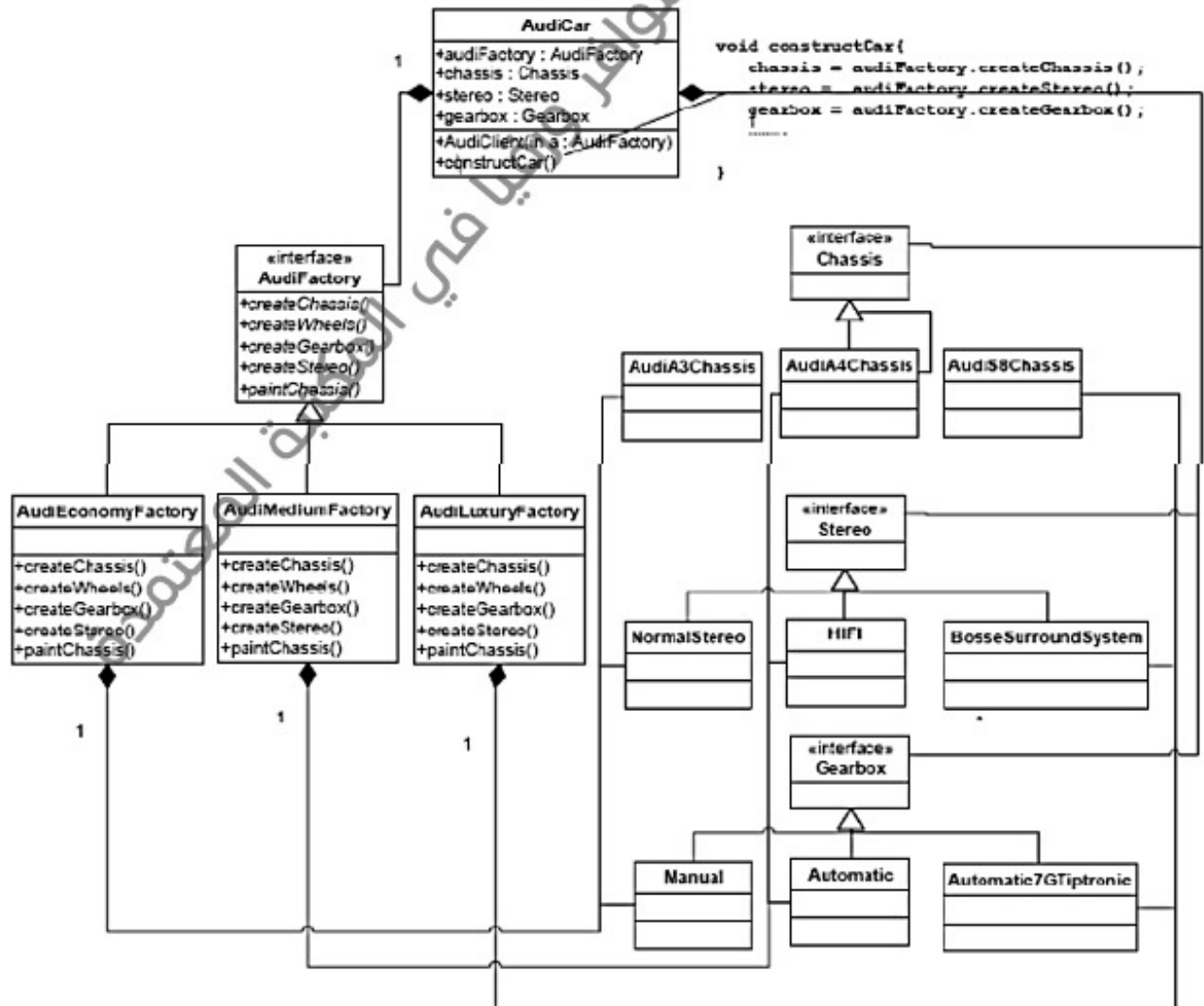
كما نعلم عن لعبة المتاهة تملك عدة مستويات ففي حال أردنا أن يكون المستوى الأول عن مرحلة مكونة من جدران وأبواب والهدف منها هو إيجاد المخرج ثم طورنا المستوى الثاني ليتألف من أبواب وجدران مفخخة.

تم تمثيل البنية السابقة ك abstract Factory حيث قمنا بتعريف interface mazeFactory، عرفنا عائلة (1) ordinaryMazeFactory.

وقمنا بتعريف عائلة (2) BombedMazeFactory، كلتا العائلتان ترثان من ال MazeFactroy، ضمن كلاس mazeGame يتم تغيير نمط المتاهة وفقاً للبارامتر الممرر للتابع CreateMaze().

مثال : سيارة الأودي

الفكرة الأساسية: لدي غرض معين مؤلف من عدة أجزاء، عندما أريد بناء هذه الأجزاء لا يجب أن أتعامل مع الصفوف الحقيقية (Chassis, Stereo, Gearbox) لذلك أقوم بعمل طبقة وسيطة وهي ال abstract factory وهو الذي يعرف ما في هذه المكونات ولكن كل مرة يقوم باستدعائهم من عائلة حسب الكلاس الذي يرث منه. الشخص الذي يقوم ببناء سيارة مؤلفة من (مكون، مكونين، ثلاثة أو أربعة) هو لا يعرف الصفوف الحقيقية إنما يطلب هذه المكونات من ال abstract factory والذي بدوره يتعامل مع الصفوف الحقيقية مما يسمح بالمستقبل إضافة عائلة جديدة من دون أي تغيير على البرنامج (وهذا يحقق مبدأ ال OCP).



نستعمل ال Abstract Factory بهذه الطريقة:

عندما نريد بناء سيارة من النوع الاقتصادي نقوم بإنشاء AudiEconomyFactory سميناه factory1.

ثم نقوم بإنشاء سيارة جديدة ونرسل هذا ال factory مع الباني الخاص بها، فيقوم بإنشاء سيارة اقتصادية

وبنفس الطريقة عندما نريد بناء سيارة فاخرة ننشئ غرض من AudiLuxuryFactory ونرسله مع الباني الخاص بالسيارة، وهكذا...

```
public class AudiTestDrive {
    public static void main(String[] args) {

        AudiCar myAudi = null;

        //create and economy car
        AudiFactory factory1 = new AudiEconomyFactory
        myAudi = new AudiCar(factory1);

        //create and medium car
        AudiFactory factory2 = new AudiMediumFactory
        myAudi = new AudiCar(factory2);

        //create and luxury car
        AudiFactory factory3 = new AudiLuxuryFactory
        myAudi = new AudiCar(factory3);

    }
}
```

Builder

يفصل عملية بناء غرض مركب (الجزء الثابت) عن عملية تمثيل هذا الغرض ويسمح لعملية البناء الواحدة بأن تعطي أكثر من تمثيل بحيث يمكن إنشاء طرق عرض عديدة بنفس خطوات عملية البناء.

مثال تحويل من word إلى pdf

وليكن مكونات ال word هي (text , image , paragraph) بفرض قمنا بكتابة البرنامج التالي للتحويل من word إلى pdf:

عندما يجد أي مكون مما سبق يقوم باستبداله إلى المقابل بال pdf

```
1 while(doc!=end){
2     getToken();
3     if(paragraph)
4         -
5         -
6     if(text)
7         -
8         -
9     if(image)
10        -
11        -
12 }
```



في حال أردنا تحويل الملف من ال word إلى HTML فسيكون هناك طريقة أخرى لعرض المكونات فإذا أردنا إجراء التحويل السابق فإننا سنقوم بقراءة هذا الكود وعند الشروط أقوم بإجراء التعديل فيما يتناسب مع ال html وليس pdf وبالتالي لم يتحقق مبدأ ال OCP.

فيقوم ال Builder بحل المشكلة عن طريق عمل واجهة ترث منها الأنواع المختلفة للتحويلات، وكلما أردنا إضافة نوع جديد فسيرث من هذه ال interface.

ال Builder يختلف عن ال Abstract Factory بوجود جزأين:

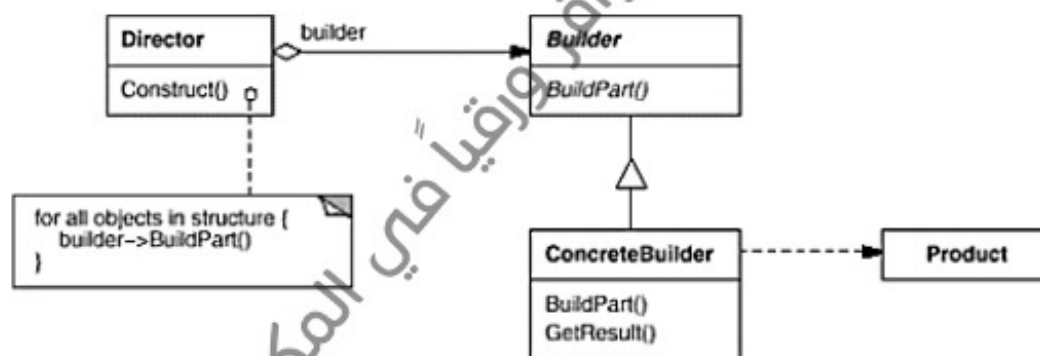
جزء ثابت (بمثالنا word) والجزء الثاني هو الذي يتغير بطريقة العرض أو التمثيل أما في ال abstract لا يوجد جزء ثابت وإنما هي عملية إضافة عائلات.

Builder أقوى و أعم من Abstract Factory و يمكن أن يحل محله.

ملاحظة:

يمكن لبعض ال design pattern أن يحل محل الأخرى أي يمكن لبعض المسائل أن تقبل حلولاً بأكثر من design pattern وهذا لا ينفي صحة أحد الحلول.

بنية ال builder:

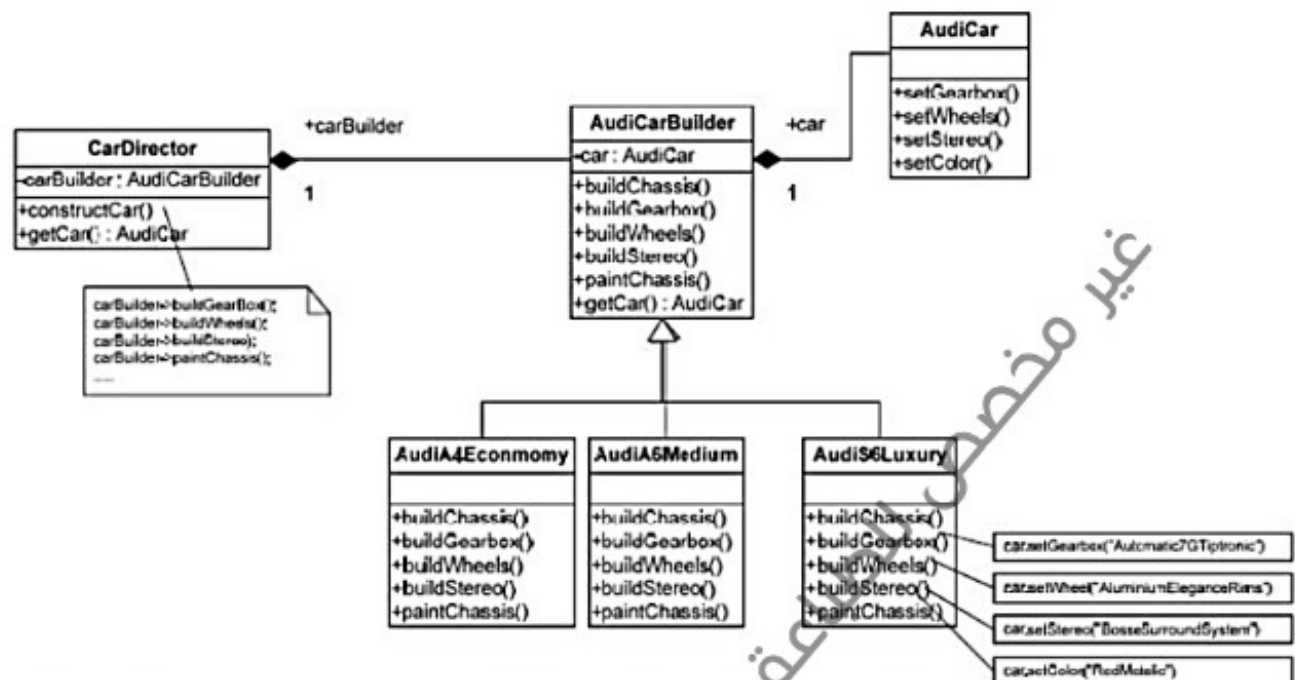


ال Director يمثل الجزء الثابت، وهو الذي يحدد الخطوات الثابتة التي يتم تنفيذها دائماً لبناء الغرض المطلوب، ولكن وفقاً لل concrete builder الذي يحتويه.

علاقته مع ال Builder علاقة Composition وهذه العلاقة مفضلة عن الوراثة لأنها تسمح بتغيير سلوك التطبيق ديناميكياً، حيث يقوم تابع ال Construct() لبناء الغرض المطلوب طبقاً للخطوات المحددة فيه، كل خطوة يكون تنفيذها بشكل مختلف وفقاً لل builder الذي يحتويه ال Director.

واختر إنجازك بناءً على المطلوب منك، لا على ما تتمناه فقط!

مثال: سيارة الأودي باستخدام ال Builder



لتحقيق ذلك برمجياً:

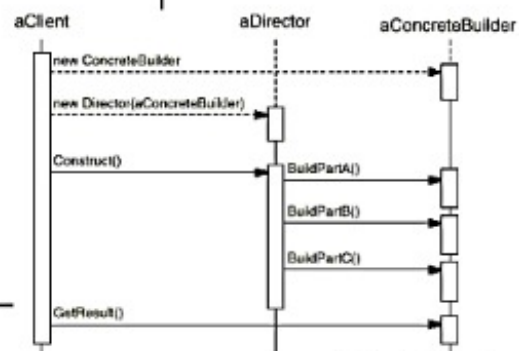
```

public class AudiCarBuilderExample {
    public static void main(String[] args) {
        CarDirector cdirector = new CarDirector();
        AudiCarBuilder a4EconomyBuilder = new AudiA4Economy();
        AudiCarBuilder a6MediumBuilder = new AudiA6Medium();
        AudiCarBuilder s6LuxuryBuilder = new AudiS6Luxury();

        cdirector.setCarBuilder(a4EconomyBuilder);
        cdirector.constructCar();
        AudiCar car = cdirector.getCar();

        cdirector.setCarBuilder(a6MediumBuilder);
        cdirector.constructCar();
        car = cdirector.getCar();

        cdirector.setCarBuilder(s6LuxuryBuilder);
        cdirector.constructCar();
        car = cdirector.getCar();
    }
}
  
```

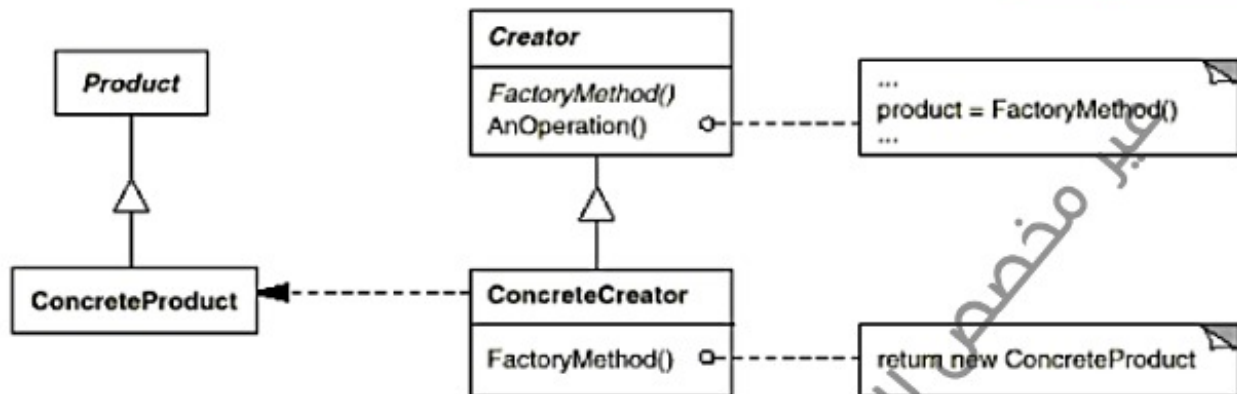


Factory Method

- يوفر واجهة لإنشاء غرض، تقوم الكلاسات الفرعية باختيار من أي كلاس ينشأ هذا الغرض (يترك قرار اختيار نوع الغرض للأبناء).
- يستخدم لتمكين البرنامج من التعامل مع ال subclasses جديدة لم تكن موجودة.
- يشابه ال Iterator في ال collection في الجافا.

مثال: ال Antivirus في نظام الويندوز قائم بتصميمه على أساس ال Factory Method حيث يكون مهياً للتعامل مع فيروسات جديدة بالتالي لسنا بحاجة لتنزيل نسخ جديدة من ال Antivirus كل مرة عند ظهور أنواع جديدة من الفيروسات.

بنية ال Factory Method



■ مثال: قبل استعمال ال factory.method

بفرض لدينا التطبيق التالي القائم على إدخال اسم الشكل فيقوم برسمه (الأشكال محدودة)، ضمن الكود التالي نعرف ال shape factory نقوم بتمرير نمط String للتابع لتحديد الشكل المراد رسمه (دائرة، مربع) بحيث يرث الصفان (circle و shape) التابعين draw , erase من كلاس shape ويتم عادة تعريفهما بما يتوافق مع بنية الكلاس كما موضح بالكود التالي:

```

public abstract class Shape {
    public abstract void draw();
    public abstract void erase();
    public static Shape factory(String type) throws BadShapeCreation {
        if(type.equals("Circle")) return new Circle();
        if(type.equals("Square")) return new Square();
        throw new BadShapeCreation(type);
    }
}

public class Circle extends Shape {
    public Circle() {}
    public void draw() { System.out.println("Circle.draw");}
    public void erase() { System.out.println("Circle.erase");}
}
  
```

OCP?

في حال أردنا تطوير النسخة السابقة من التطبيق ليشمل إضافة شكل النجمة إلى مجموعة الأشكال السابقة فإن الحل المقترح سيكون بإضافة كلاس star يرث من صف ال shape إضافة لذلك نضيف ضمن التابع shape factory الشرط التالي:

if (type.equals("Star")) return new star();

وهذه مخالف لمبدأ ال OCP.

فتم التعديل على الكود واستخدام factory method كالتالي:

إنشاء واجهة ShapeFactory ترث منها كلاسات فرعية وفق الأشكال المطلوبة (CircleFactory،

RectangleFactory، ...) تطبيق تابع createShape بإنشاء الشكل المناسب،

وضمن صف AllshapeFactories قمنا بتعريف map تربط اسم الشكل ك نمط string مع ال ShapeFactory،

وضمن ال createShape الخاص بها نستفيد من بنية ال map للحصول على ال object من نمط الشكل المطلوب

فرضا Circle واستدعاء ال create shape من أجله.

أثناء التنفيذ ضمن صف Circle فإنه يقوم بتنفيذ جزء ال static من البنية حيث يقوم بتحميل , name

circlefactory فوراً إلى ال map كما موضح بالكود التالي:

```
public interface ShapeFactory {
    public Shape createShape();
}
public class AllShapeFactories {
    public static Map<String, ShapeFactory> allFactories = new
        HashMap<String, ShapeFactory>();
    public static final Shape createShape(String id) {
        // "throw BadShapeCreation"
        return allFactories.get(id).createShape();
    }
}
public class Circle extends Shape {
    static { AllShapeFactories.allFactories.put("Circle", new CircleFactory()); }
    (...)
}
public class CircleFactory implements ShapeFactory {
    public Shape createShape() { return new Circle(); }
}
public class Rectangle extends Shape {
    static { AllShapeFactories.allFactories.put("Rectangle", new RectangleFactory()); }
    (...)
}
public class RectangleFactory implements ShapeFactory {
    public Shape createShape() { return new Rectangle(); }
}
```

إذا أردنا إضافة شكل جديد فما علينا إلا أن نعرف صف باسم الشكل الجديد Star بالإضافة إلى StarFactory يرث من

ShapeFactory ويطبق التابع createShape ليقوم بإنشاء غرض من ال Star، لكن في مثل هذه الحالة فنحن بحاجة

إلى إعادة إقلاع الجهاز من أجل تحميل الأشكال الجديدة إلى الذاكرة أما إذا أردنا الإيعيد الإقلاع ويتحمل ديناميكيًا فيتم

بالشكل التالي:

```
public interface Shape {
    public void draw();
    public void erase();
}
public abstract class ShapeFactory {
    protected abstract Shape create();
    static Map<String, ShapeFactory> factories = new HashMap<String, ShapeFactory>();
    // A Template Method:
    public static final Shape createShape(String id) throws BadShapeCreation {
        if(!factories.containsKey(id)) {
            try { Class.forName(id); }
            catch(ClassNotFoundException e) { throw new BadShapeCreation(id); }
            if(!factories.containsKey(id)) throw new BadShapeCreation(id);
        }
        return factories.get(id).create();
    }
}
public class Circle implements Shape {
    private Circle() {}
    public void draw() { System.out.println("Circle.draw"); }
    public void erase() { System.out.println("Circle.erase"); }
    private static class Factory extends ShapeFactory {
        protected Shape create() { return new Circle(); }
    }
    static { ShapeFactory.factories.put("Circle", new Circle.Factory()); }
}
```

■ مثال: حل مسألة المتاهة بطريقة ال Factory Method

```

class MazeGame {
    Maze MakeMaze() { return new Maze(); }
    Wall MakeWall() { return new Wall(); }
    Room MakeRoom(int n) { return new Room(n); }
    Door MakeDoor(Room r1, Room r2)
    { return new Door(r1, r2); }

    Maze CreateMaze () {
        Maze aMaze = MakeMaze();
        Room r1 = MakeRoom(1);
        Room r2 = MakeRoom(2);
        Door aDoor = MakeDoor(r1, r2);
        aMaze.AddRoom(r1);
        aMaze.AddRoom(r2);

        // ...
        return aMaze;
    }
}

```

```

class BombedMazeGame extends MazeGame {
    Wall MakeWall() { return new BombedWall(); }
    Room MakeRoom(int n)
    { return new RoomWithBombe(n); }
}

```

```

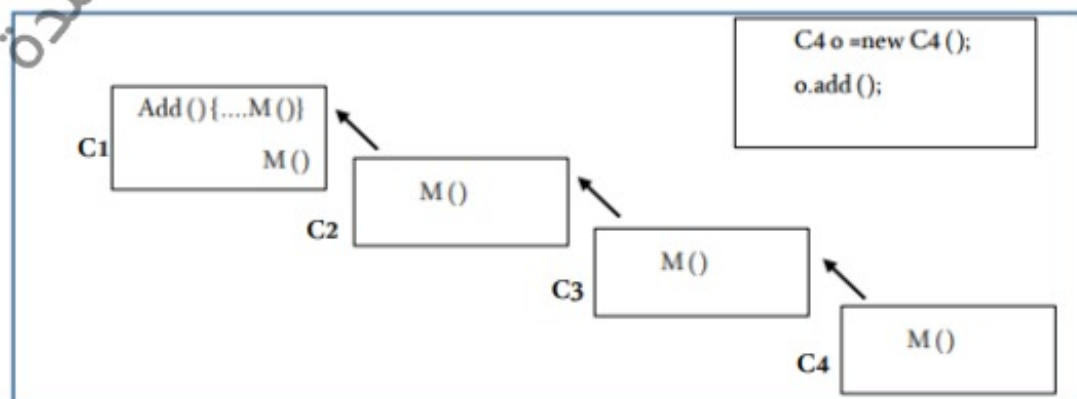
class EnchantedMazeGame extends MazeGame {
    Room MakeRoom(int n)
    { return new EnchantedRoom(n); }
    Door MakeDoor(Room r1, Room r2)
    { return new DoorNeedingSpell(r1, r2); }
}

```

ال factory method يستخدم عملية الوراثة، ضمن الكلاس MazeGame نعرف البنية الأساسية للمستوى الأول من لعبة المتاهة (Maze , Room , Wall , Door) ، في المستوى الثاني (الجدران مفخخة) الصف BombedMazeGam يرث من MazeGame ويعيد تعريف بنية ال Wall و ال Room ويستدعي التابع createMaze من كلاس MazeGame ، وكذلك الأمر بالنسبة ل EnchantedMazeGame.

ملاحظة 1:

قاعدة الربط الديناميكي تنص على أنه يتم استدعاء ال Method من الكلاس الأقرب لهذا الغرض، على فرض قمنا بتعريف abstract method ضمن الكلاس c1 وكانت الصفوف c2, c3, c4 ترث من c1 وضمن الكلاس c4 قمنا باستدعاء ال add، في حال لم تكن معرفة ضمن ال c4 يقوم الأب c1 بتنفيذ ال add ولكن لا ينفذ ال m() ال abstract وإنما يقوم بتنفيذ ال m() الموجودة بالصف c4، في حال لم يجدها يستدعيها من الصف الأعلى منه c3 كما موضح بالشكل التالي:



ملاحظة 2:

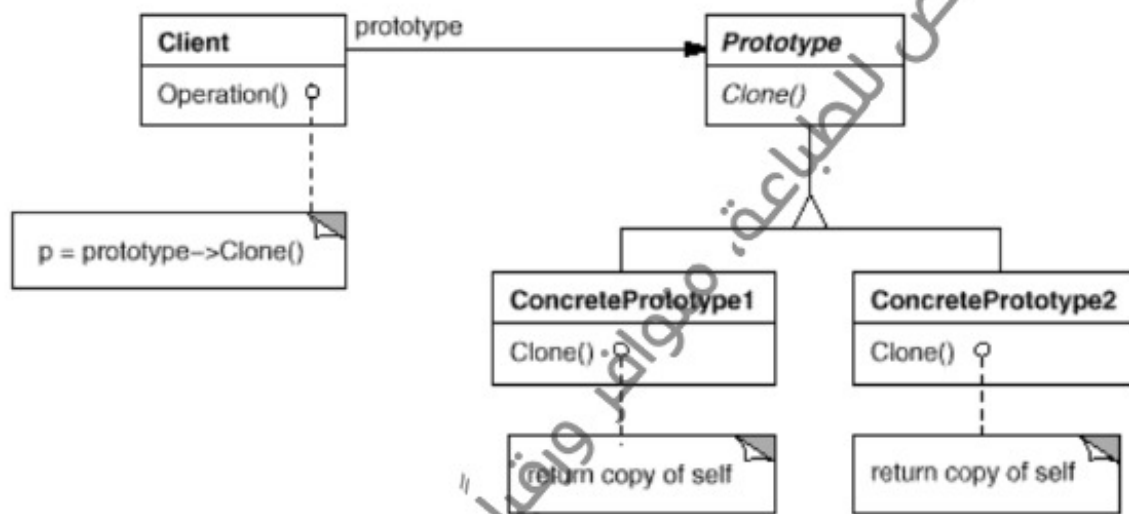
بالنماذج التصميمية الـ 23، فقط اثنان يستخدمان الوراثة وهما (Adapter & Factory Method) والباقي يستخدمون الـ Composition.

Prototype

Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.

يوجد عدة نماذج أولية مختلفة والتي يتم نسخهم وتجميعهم.

هرمية ال Prototype:



شرح بنية Prototype

يمثل Client النظام أو التطبيق، بينما Prototype يمثل كلاس الاب الذي سنرث منه، يحوي تابع clone() و الذي سنقوم بنسخ محتوياته في الكلاسات الوراثة منه التي تمثل ConcretePrototype1 و ConcretePrototype2.

مثال المتاهة باستخدام ال Prototype :

```

class MazePrototypeFactory extends MazeFactory {
    private Maze _prototypeMaze;
    private Room _prototypeRoom;
    private Wall _prototypeWall;
    private Door _prototypeDoor;

    public MazePrototypeFactory(Maze m, Wall w, Room r, Door d) {
        _prototypeMaze = m; _prototypeRoom = r; _prototypeWall = w; _prototypeDoor = d;
    }

    Wall makeWall() {return (Wall) _prototypeWall.clone();}
    Door makeDoor(Room r1, Room r2) {
        Door d = (Door) _prototypeDoor.clone();
        d.initialize(r1, r2);
        return d;
    }
    Maze makeMaze() { ..... }
    Room makeRoom(int) { ..... }
    ....
}
  
```

يمثل MazePrototypeFactory الكلاس الذي سنقوم بنسخ محتوياته عن طريق التابع clone واستخدامهم في كلاس آخر.

ملاحظة :

عندما يتم تنفيذ المنهج clone() والموروث من الصف Object في الجافا سيكون هناك مشكلة حتما عندما يحوي الصف على أعضاء غير بدائيين (مراجع على أغراض أخرى) ولذلك:

```
public Object deepClone() {
    try {
        ByteArrayOutputStream b = new ByteArrayOutputStream();
        ObjectOutputStream out = new ObjectOutputStream(b);
        out.writeObject(this);
        ByteArrayInputStream bIn = new ByteArrayInputStream(b.toByteArray());
        ObjectInputStream oi = new ObjectInputStream(bIn);
        return (oi.readObject());
    }
    catch (Exception e)
    {
        System.out.println("exception:" + e.getMessage());
        return null;
    }
}
```

Structural pattern

تهتم بتجميع الصفوف و الأغراض لتشكيل بنى أوسع ، سنأخذ أنواع ضمنه:

- Composite
- Decorator
- Proxy
- Adapter
- Bridge
- Flyweight

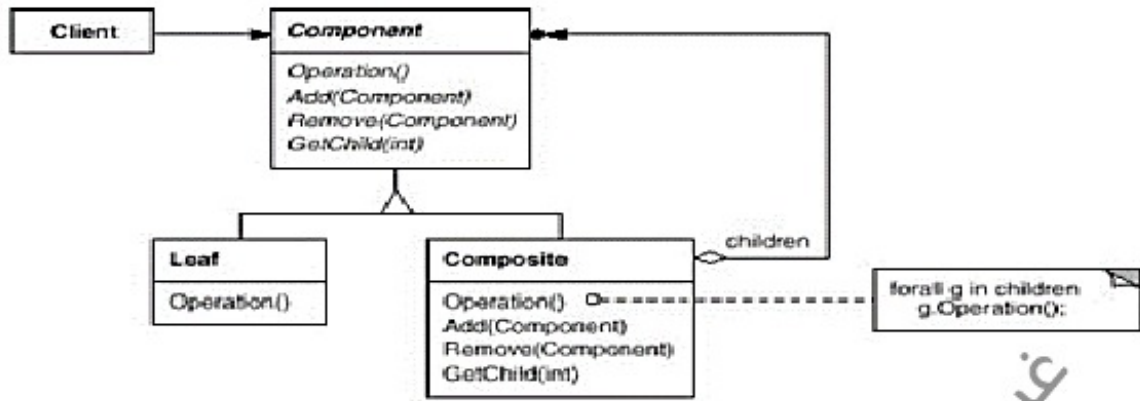


Composite

توصيف: تحدد كيف يتم تجميع الأغراض.

البنية العامة: يتألف من client يتخاطب مع component يتفرع الى leaf و composite و ال composite

يتفرع ل leaf و composite (عملية تعاوديه) وذلك باحتوائه على مصفوفة Components، ضمن ال leaf نعرف العملية الخاصة بالورقة ، و ضمن ال composite و ال component نعرف العمليات الأساسية القائمة بإضافة component جديد ، و حذف ال component او الحصول على أبناء هذا ال component كما موضح بالهرمية التالية:

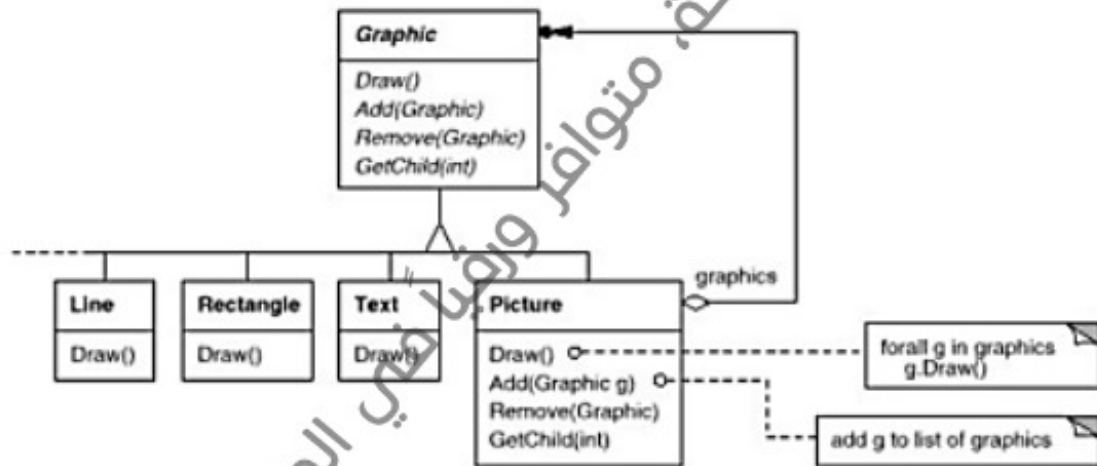


5

مثال :

عائلة ال Swing في الجافا

يمكن ل Graphic أن يتألف من عناصر بسيطة (Line , Text) و عناصر مركبة (picture) و هذه الأخيرة تتكون من العناصر البسيطة السابقة ، فظهرت العودية ببنية ال picture .

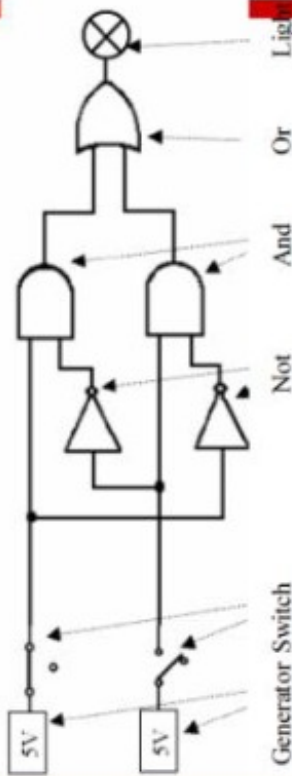
**مثال :**

برنامج الدارات المنطقية :

نلاحظ ال components الأساسية بالبنية هي :

- منبع الطاقة (البطارية)
- Not (لقلب الإشارة)
- And , Or (تملك مدخلين دخل)
- Light (الضوء)
- Switch (open , close)

لا بُدَ لَكَ واحدٍ مِنَّا أَنْ يُسَجِّلَ يَوْمًا فاصِلًا ،
يَكُونُ مُخْتَلَفًا بَعْدَهُ .



نريد نمذجة دارات منطقية اعتباراً من Components كما في الشكل التالي :

الـ Component يملك على الأكثر دخلين و خرج وحيد.

من أجل بناء دارة فإن خرج Component يجب أن يتصل إلى مدخل Component .

خرج الـ Component يمكن أن يكون فعالاً أو لا. هذا يتوقف على نوع الـ Component و قيم دخله.

نقول إن دخلاً فعالاً إذا كان موصولاً إلى خرج فعال و غير فعال إذا كان موصولاً إلى خرج غير فعال.

الـ Generator لا يملك دخلاً و خرجة نوعاً فعالاً.

الـ Light لا يملك خرجاً و هي مضيئة إذا كانت فعالة.

الـ Switch يمكن أن يكون مفتوحاً {غير فعال} أو موصولاً {فعالاً}.

الـ Not و الـ Switch هما بخرج وحيد و دخل وحيد.

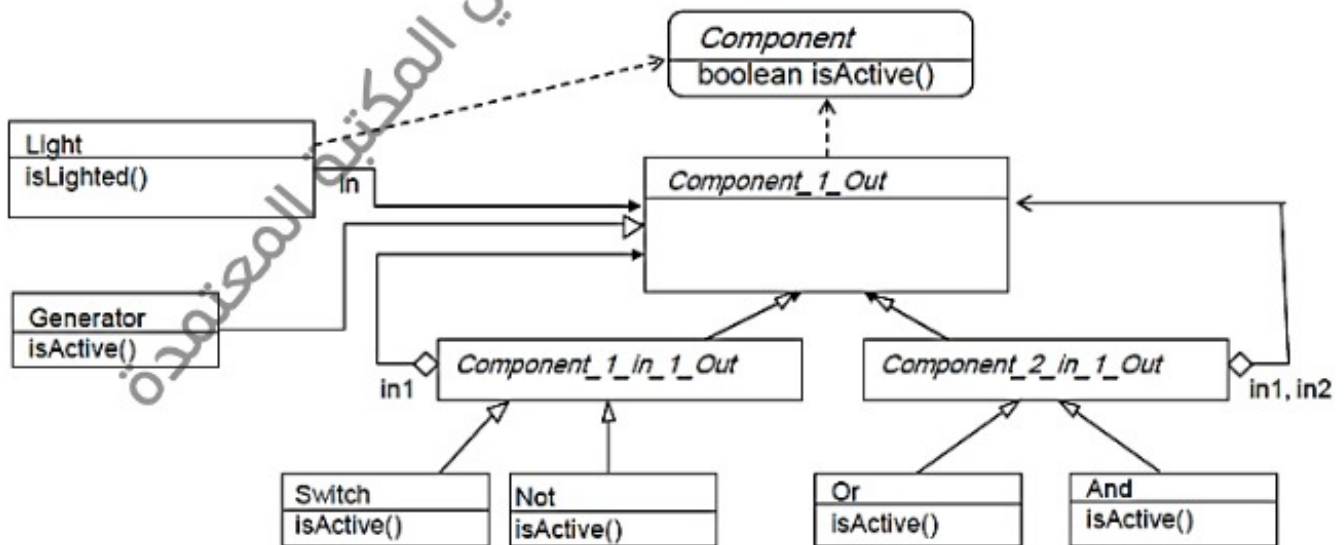
يمكننا أن نرى أيضاً Components بدخلين و خرج وحيد كـ الـ And و الـ Or.

دخل الـ Component يتحقق هوامياً بالـ Component المتصل به. إذا لم يتصل الدخل بـ

Component فهو null. إذا أردنا حساب خرج الـ Component الذي يملك دخلاً null

فإنه يجب رفع Exception NullPointerException.

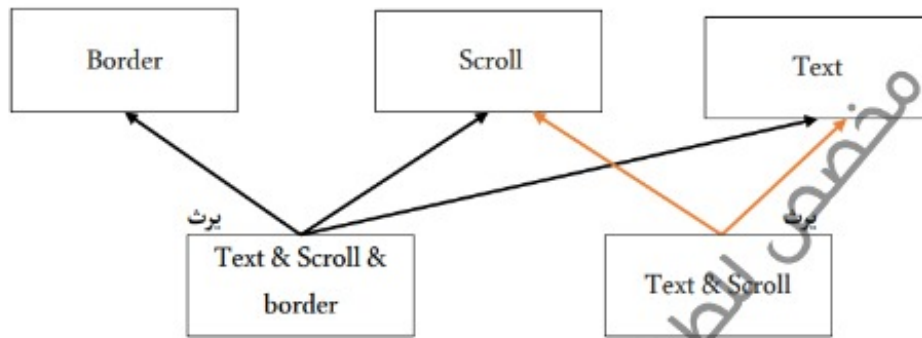
عرفنا بالهرمية التالية component_1_out التي تتألف من مخرج وحيد يرث من هذا المكون مجموعتين و هما component_1_in_1_out مثل الـ (switch , not) و component_2_in_1_out مثل الـ (or , and) يمكن اعتبار شرط التوقف light (is Lighted())



Decorator

يضيف فعالية جديدة لغرض معين بشكل ديناميكي أثناء التنفيذ أي دون تعديل الغرض الأساسي و يبقى على الواجهة الخاصة بهذا الغرض كما هي.

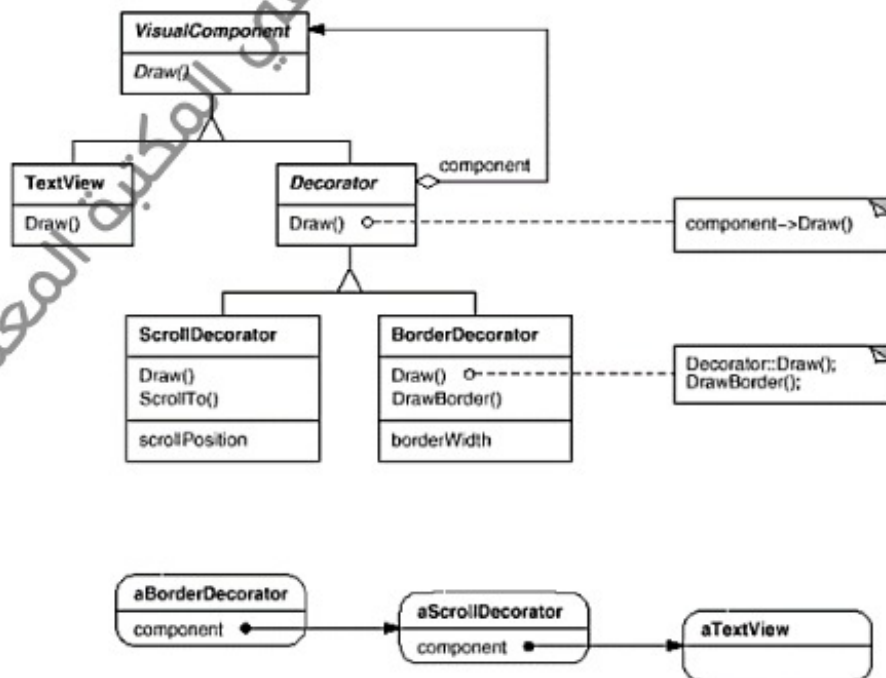
- مثال : I/O Stream في الجافا
- مثال : لدي مكونات صفحة معينة هي text , scroll , border



و هكذا مع كثرة العناصر تكثر احتمالات انشاء الأغراض مما يؤدي الى انفجار بعدد الصفوف

هل بالإمكان حل هذه المشكلة بدون انشاء أصناف جديدة او بعدد أصناف ثابت و تتم عملية الإضافة الدائمة عليها؟

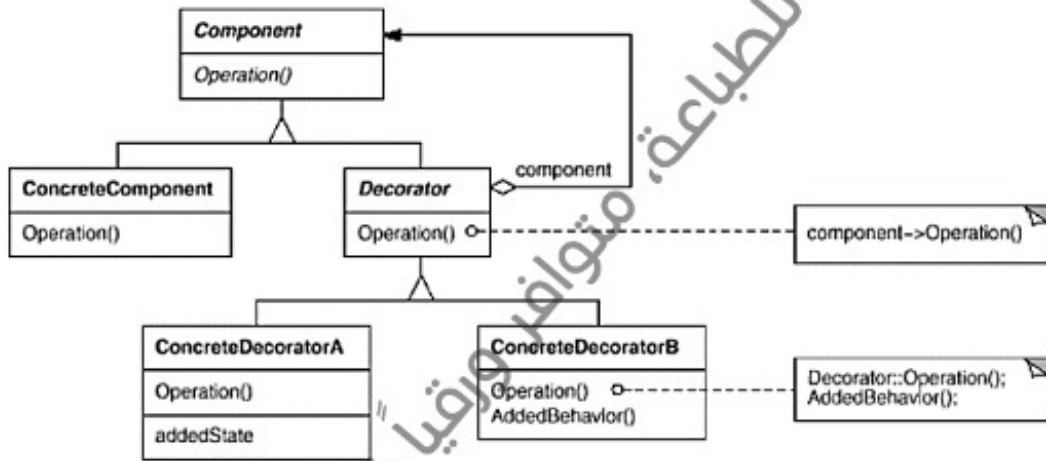
يمكن ان يتم ذلك عن طريق الاستعاضة عن عملية الوراثة الموجودة بعملية ال composition :



- ال Decorator هو تطوير على composite بحيث نعتبر ال Text هو Leaf و يرث من Visual Component عملية ال Draw().
- نعتبر أيضا Decorator هو ال composite يرث من هذا الصف ال ScrollDecorator و ال BorderDecorator و ضمن هذين الصفين نرث من صف ال Decorator عملية Draw().
- إضافة على ذلك نعرف عمليات إضافية خاصة بكل منهما مثل Scroll To() خاص بالصف ScrollDecorator و DrawBorder() خاص بالصف BorderDecorator.
- تسلسل تنفيذ عملية ال Draw() ينفذ أولا عند ال ab و من ثم إضافة العمليات الموجودة .

و بهذه الطريقة يستطيع أي Decorator ضم أي Decorator سابق فيجعل بذلك الأغراض و كأنها ترث من بعضها البعض لأنها تقوم بعملية Composition لبعضها البعض.

بنية ال Decorator



THE END

